

Security Review

Cantina Managed

COINBASE: PRECOMPILES DIFF

Cantina Managed review by:

Phaze, Lead Security Researcher

Slowfi, Lead Security Researcher

Oxhuy0512, Security Researcher

Jay, Security Researcher



September 10, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	5
3.1	High Risk	5
3.1.1	Frozen Stablecoin V1 changed <code>updatePolicy</code> access order	5
3.2	Medium Risk	5
3.2.1	Aliased owned ABI decoding is performed twice during version validation	5
3.3	Low Risk	6
3.3.1	Cobalt factory bootstrap routes token initialization through V1	6
3.3.2	Activation Registry Precompile Accepts and Strands ETH	6
3.3.3	V2 transfer <code>SLOADs</code> policy IDs before rejecting zero receiver	7
3.3.4	B-20 permit hashing and signature recovery are undercharged	7
3.4	Informational	8
3.4.1	Asset golden tests use a nonproduction multiplier prestate	8
3.4.2	Cobalt golden tests run Legacy storage features	8
3.4.3	Stablecoin V2 has no golden suite	8
3.4.4	Inaccurate comment claims no directly installed precompiles are observed	9
3.4.5	Unused ordering derives on <code>StablecoinVersion</code>	9
3.4.6	Inverted seizable wording on <code>ensure_seizable</code> and related comments	9
3.4.7	Cobalt composite calls and errors are missing from metrics	10
3.4.8	Zenith activation is described incorrectly	10
3.4.9	Cobalt only wrappers silently select Legacy storage features	10
3.4.10	Public nonce increment helper trusts stale caller state	11
3.4.11	Direct provider use can omit fork installed precompiles	11
3.4.12	Malformed dynamic array lengths can change ABI failure behavior by allocator	11
3.4.13	Static context mutations return catchable reverts instead of exceptional halts	12
3.4.14	Full calldata is copied before gas admission	13



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Aug 17th to Sep 2nd the Cantina team conducted a review of [base](#) on commit hash [ceebc29e](#). The team identified a total of **20** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	1	1	0
Low Risk	4	4	0
Gas Optimizations	0	0	0
Informational	14	11	3
Total	20	17	3

2.1 Scope

The security review had the following components in scope for [base](#) on commit hash [ceebc29e](#):

```
crates/common/precompiles/src
├── lib.rs
├── macros.rs
├── provider.rs
├── spec.rs
├── activation
│   └── precompile.rs
├── b20_asset
│   ├── accounting.rs
│   ├── dispatch.rs
│   ├── mod.rs
│   ├── precompile.rs
│   ├── storage.rs
│   ├── versions.rs
│   ├── abi
│   │   ├── mod.rs
│   │   └── v2.rs
│   └── logic
│       ├── interface.rs
│       ├── mod.rs
│       ├── v1.rs
│       └── v2.rs
├── b20_factory
│   ├── abi
│   │   └── mod.rs
│   └── logic
│       ├── interface.rs
│       ├── mod.rs
│       └── v1.rs
├── b20_stablecoin
│   ├── dispatch.rs
│   ├── mod.rs
│   ├── precompile.rs
│   ├── versions.rs
│   └── logic
│       ├── interface.rs
│       └── mod.rs
```



```
├── v1.rs
├── v2.rs
├── common
│   ├── abi_fingerprint.rs
│   ├── core_storage.rs
│   ├── mod.rs
│   ├── pausable_feature.rs
│   ├── policy_type.rs
│   ├── token.rs
│   ├── token_accounting.rs
│   ├── abi
│   │   ├── b20_abi.rs
│   │   ├── mod.rs
│   │   └── v2.rs
│   └── ops
│       ├── guards.rs
│       ├── mod.rs
│       ├── permit.rs
│       └── roles.rs
├── nonce
│   ├── abi.rs
│   ├── dispatch.rs
│   └── storage.rs
├── policy
│   ├── accounting.rs
│   ├── dispatch.rs
│   ├── mod.rs
│   ├── precompile.rs
│   ├── storage.rs
│   ├── versions.rs
│   ├── abi
│   │   ├── mod.rs
│   │   └── v2.rs
│   └── logic
│       ├── interface.rs
│       ├── mod.rs
│       ├── v1.rs
│       └── v2.rs
└── tx_context
    ├── abi.rs
    ├── dispatch.rs
    └── storage.rs
```



3 Findings

3.1 High Risk

3.1.1 Frozen Stablecoin V1 changed `updatePolicy` access order

Severity: High Risk.

Context: [v1.rs#L556-L574](#).

Description: The deployed Beryl Stablecoin V1 reads the current `policyId` before checking `policyExists(newPolicyId)`, while this implementation checks the new policy first and skips the old policy `SLOAD` when the ID is invalid. That changes gas and caller visible revert behavior for the frozen Beryl surface.

Triggering the divergence requires an authorized call to `updatePolicy(TRANSFER_SENDER_POLICY, 99)` with about `3,000` forwarded gas: the reference returns `OutOfGas` with empty data, while the current implementation returns typed `PolicyNotFound(99)` with `882` gas left, and the divergence window is `2,118` to `4,217` gas.

For the frozen Beryl surface, callers can observe a different halt or revert class, returndata, gas behavior, and storage access compared with the deployed reference. A caller with a constrained gas budget can receive `OutOfGas` instead of the reference result.

Recommendation: Restore old policy first ordering in Stablecoin V1 and V2 and add differential golden tests for halt or revert class, returndata, gas, storage accesses, logs, and state root.

Coinbase: Fixed in [PR 4596](#).

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Aliased owned ABI decoding is performed twice during version validation

Severity: Medium Risk.

Context: [versions.rs#L89-L121](#), [b20_abi.rs#L35-L62](#), [versions.rs#L72-L108](#).

Description: The exercised `announce(bytes[] internalCalls,string id,string description,string uri)` payload used `1,024` `bytes[]` entries whose offsets all referenced the same `65,536` byte tail. Version validation allocates an owned value and canonical dispatch allocates it again. The full route therefore doubles the owned decoding work and remains substantially under metered for aliased dynamic data.

A single successful protocol capped transaction occupied the measured release executor for `94.39` seconds with the ordinary system allocator and generated approximately `97.9 GiB` of cumulative allocation requests. The demonstrated impact is executor CPU saturation and allocator churn, not simultaneous resident memory of that size.

Permissionless access is possible through a contract that copies the valid aliased payload once and repeatedly calls the precompile in one transaction.

Recommendation: Bound dynamic depth, element counts, and aggregate expanded decoded bytes before owned materialization; meter traversed offsets, elements, and expanded bytes; decode once into a routable versioned value or make the validation borrowed and allocation free. Add adversarial aliased tail, nesting, overlap, malformed offset, and near limit tests.

Coinbase: Fixed in [PR 4954](#).

Cantina Managed: Fix verified.



3.3 Low Risk

3.3.1 Cobalt factory bootstrap routes token initialization through V1

Severity: Low Risk.

Context: [versions.rs#L109-L114](#).

Description: At `Cobalt`, `FactoryVersion` remains `V1` and its bootstrap path dispatches `Asset` and `Stablecoin` init calls through token `V1` instead of resolving the active token version. A valid Cobalt bootstrap using `updatePolicy(SEIZE_HOLDER_POLICY, compositeId)` is rejected by `Asset V1` with `UnsupportedPolicyType` and creation rolls back.

Triggering the issue requires creating valid simple policies and a `UNION` composite, then creating an adminless `Asset` with an init call that updates `SEIZE_HOLDER_POLICY` to the composite ID; the pinned base standard reference succeeds while native `Factory V1` rejects the call.

Valid Cobalt bootstrap transactions that depend on newer policy types can revert and roll back, preventing expected token creation.

Recommendation: Resolve the active `Asset` and `Stablecoin` version for bootstrap init calls at `Cobalt` while preserving the required bootstrap privilege and add a Cobalt factory golden for this path.

Coinbase: Fixed in [PR 4598](#).

Cantina Managed: [PR 4598](#) updates the factory init dispatch to resolve the active token version dynamically using `PolicyVersions::from_base_upgrade(upgrade)` rather than routing all bootstrap init calls through the hardcoded `V1` path, allowing Cobalt bootstrap transactions to use the correct policy types without requiring a new `FactoryVersion`.

3.3.2 Activation Registry Precompile Accepts and Strands ETH

Severity: Low Risk.

Context: [precompile.rs#L72-L78](#).

Description: The activation entry point hands calldata straight into `dispatch_with_observer`, and that dispatcher moves immediately to gas accounting and selector routing without ever checking whether value was attached to the call. The shared `base_precompile` macro only rejects `delegatecall` through its direct call guard and does nothing about value, so the value path stays wide open. Because the read only views `isActivated` and `checkActivated` require no authorization, any caller can reach a successful dispatch while carrying ETH, at which point the value is transferred to the precompile address and left there with no bytecode able to recover it, making the loss permanent. This also breaks a crate-wide invariant since every other precompile such as `b20_asset`, `policy`, `tx_context`, `b20_stablecoin`, `b20_factory`, and `nonce` rejects `nonzero` value with `NonPayable` at the top of dispatch, while activation is the sole exception and `IActivationRegistry` doesn't seem to ever define a `NonPayable` error in itself.

Recommendation: Add a `NonPayable` error to the `IActivationRegistry` interface and reject `nonzero ctx.call_value` at the top of `dispatch_with_observer` before any gas is charged or routing occurs, gated on `BaseUpgrade` so it fires only at and after Cobalt while pre fork replay stays byte identical. Thread the active upgrade into the dispatcher, since it is currently in scope at `precompile_with_observer` but not passed down. Pair this with a version gated test modeled on `dispatch_rejects_call_with_nonzero_value` asserting that pre fork the value call still succeeds and post fork it reverts with the encoded `NonPayable` error, restoring the crate wide `nonpayable` invariant.

Coinbase: Fixed in [PR 4808](#).

Cantina Managed: Fix verified.



3.3.3 V2 transfer SLOADs policy IDs before rejecting zero receiver

Severity: Low Risk.

Context: v2.rs#L73-L77, v2.rs#L240-L254, v2.rs#L51-L55, v2.rs#L194-L207.

Description: In the non-privileged path of `AssetV2` and `StablecoinV2` `transfer` and `transferWithMemo` the code loads transfer policy IDs with one cold `SLOAD` before the zero receiver and zero sender checks run inside `transfer_inner`. The Cobalt base-std mock and its revert order suite require pause then zero receiver then zero sender then policy then balance, and frozen V1 plus V2 `transfer_from` already reject a zero address before any policy read. On a reject path that always reverts `InvalidReceiver` or `InvalidSender`, V2 pays one extra cold `SLOAD` of about 2100 gas that frozen V1 and the mock do not, since they revert before any policy read, and under a tight stipend the reference returns a typed revert while V2 can hit `OutOfGas` with empty return data. Behavior matches under normal gas so severity is low, but this is a genuine storage access divergence from the Solidity reference this repo claims to match on `SLOAD` and `SSTORE` counts. Existing goldens miss it because `golden_transfer_reverts_zero_receiver` uses the privileged path which never loads policies, `golden_gas_footprints` measures only the success path, and the base-std revert order tests assert which error wins not how many `SLOAD`s ran.

Recommendation: Lift the zero receiver and zero sender checks above `transfer_policy_ids` in both `AssetV2` and `StablecoinV2` `transfer` so the order matches `transfer_from`, frozen V1, and the Cobalt mock. `transferWithMemo` routes through `transfer` so the same change covers it. Add a gas and storage access golden for the non-privileged zero address reject path to lock the `SLOAD` count.

Coinbase: Fixed in [PR 4823](#).

Cantina Managed: Fix verified.

3.3.4 B-20 permit hashing and signature recovery are undercharged

Severity: Low Risk.

Context: v1.rs#L607-L625, v1.rs#L585-L603, permit.rs#L48-L101, metrics.rs#L590-L597.

Description: The four B-20 `permit` implementations execute EIP-712 hashing and secp256k1 signer recovery through `common/ops/permit.rs`. The dispatcher charges only `ceil(calldata length / 32) * 6` for the input, while the hashing and recovery helpers receive no gas context or equivalent native charge.

A contract can repeatedly call `permit` with `deadline = type(uint256).max`, `v = 27`, recoverable shaped `r` and `s`, and an unrelated `owner`. Each child performs signature recovery and then reverts with `InvalidSigner`; the caller can catch each revert and continue. No valid owner signature or token balance is required.

The remote review reproduced identical gas for a control that exits on invalid `v` and a path that performs recovery, while the recovery path took materially longer. The behavior is inherited from the baseline and affects the Beryl and Cobalt token surfaces.

A permissionless caller can put more hashing and elliptic curve work into a transaction than the native gas charge represents. This can increase validator CPU time, delay block execution, and reduce throughput. The transaction gas limit keeps the work finite, and the review found no authorization bypass, funds loss, state corruption, or consensus split.

Recommendation: Charge a fixed recovery cost comparable to the EVM `ECRECOVER` cost and charge the Keccak work, including the runtime version hash, before executing it. Apply the same accounting consistently to Asset and Stablecoin V1 and V2, then add caught revert loop and exact gas boundary tests.

Coinbase: Fixed in [PR 4891](#).



Cantina Managed: Fix verified.

3.4 Informational

3.4.1 Asset golden tests use a nonproduction multiplier prestate

Severity: Informational.

Context: [storage.rs#L265-L275](#).

Description: This test proves factory compatible storage behavior: the multiplier slot is physically `0` while the getter returns `WAD`. The Asset V1 and V2 golden harnesses preseed `WAD` instead, so they skip the factory created zero slot first write path.

Triggering the missed path requires comparing the first multiplier write from a physical `0` slot with the same write from a physical `WAD` slot; zero to nonzero uses `20,000 SSTORE` gas while nonzero to nonzero uses `2,900`, a `17,100` gas difference.

The golden tests can report matching behavior while missing the factory state and gas path used by production. Regressions in the first multiplier write can therefore pass without detection.

Recommendation: Initialize the Asset golden providers with the factory physical zero multiplier slot and pin exact gas, storage, event, and state root outputs for both versions.

Coinbase: Fixed in [PR 4601](#).

Cantina Managed: Fix verified.

3.4.2 Cobalt golden tests run Legacy storage features

Severity: Informational.

Context: [spec.rs#L9-L15](#).

Description: `UpgradeGatedStorageFeatures` maps `Cobalt` to `StorageFeatures::Cobalt`, but the Asset V2 and Policy V2 golden harnesses build `HashMapStorageProvider` with its `Legacy` default even when dispatching `BaseUpgrade::Cobalt`. Those goldens therefore do not exercise production Cobalt `SLOAD` behavior.

Triggering the test gap requires running the same Asset V2 and Policy V2 golden cases with the default provider and with `set_storage_features(StorageFeatures::Cobalt)`; the production configuration can change dynamic tail cleanup, gas, and state root results.

The suite can pass while failing to detect regressions in Cobalt storage behavior, gas, dynamic tail cleanup, or state roots.

Recommendation: Make every Cobalt golden explicitly use `StorageFeatures::Cobalt` and retain separate Legacy goldens only for behavior before Cobalt.

Coinbase: Fixed in [PR 4608](#).

Cantina Managed: Fix verified.

3.4.3 Stablecoin V2 has no golden suite

Severity: Informational.

Context: [v2.rs#L747-L748](#).



Description: This module contains unit tests, but there is no end to end Cobalt golden suite for Stablecoin V2. As a result, ABI routing, seizure behavior, typed errors, gas, storage, events, and state roots are not pinned against the deployed or reference behavior.

Triggering the coverage gap requires dispatching every V2 ABI operation through the real precompile and comparing `success` or `revert` bytes, gas, storage, events, and state roots with the pinned base standard reference.

Behavior regressions in Stablecoin V2 can reach the Cobalt implementation without a reference comparison that detects them.

Recommendation: Add and gate a Stablecoin V2 golden suite before treating the Cobalt implementation as behavior locked.

Coinbase: Fixed in [PR 4648](#).

Cantina Managed: Fix verified.

3.4.4 Inaccurate comment claims no directly installed precompiles are observed

Severity: Informational.

Context: [provider.rs#L207](#).

Description: The comment above the `Beryl` install block is wrong. It states that the observer applies only to dynamic, address derived B20 token precompiles resolved through `BerylLookup`, and that every directly installed precompile uses plain install and is therefore not observed. The four `Beryl` precompiles installed immediately after that comment all call `install_with_observer` and pass the real observer through. That observer is used by `record_b20_created`, `record_internal_calls`, and `BerylCallRecorder::start`. Only the Cobalt `TxContext` and `NonceManager` precompiles use plain install.

Recommendation: Rewrite the comment so it matches the install path. State that `Beryl` factory, lookup, policy registry, and activation registry receive the observer and emit metrics, and that only the `Cobalt` precompiles install without an observer.

Coinbase: Fixed in [PR 4559](#).

Cantina Managed: Fix verified.

3.4.5 Unused ordering derives on `StablecoinVersion`

Severity: Informational.

Context: [versions.rs#L18-L22](#).

Description: The `StablecoinVersion` enum derives `PartialOrd` and `Ord`, and the comment says the derived ordering is chronological because variants are declared in activation order. That is wrong as a reason to keep the derives. `PolicyVersion` and `AssetVersion` both stop at `PartialEq` and `Eq`, and nothing compares `StablecoinVersion` values. Routing uses `BaseUpgrade` ordering plus match self, so the ordering is unused surface that makes variant reordering silently meaningful.

Recommendation: Drop `PartialOrd` and `Ord` so `StablecoinVersion` matches the sibling version enums, and remove the line about chronological derived ordering.

Coinbase: Fixed in [PR 4559](#).

Cantina Managed: Fix verified.

3.4.6 Inverted seizable wording on `ensure_seizable` and related comments

Severity: Informational.

Context: [guards.rs#L74-L79](#).



Description: The first line of `ensure_seizable` looks to be incorrect. It says an account is seizable when it is a member of the current seize holder policy, which reads as membership means seizable. The check and the reference mock treat it the other way around: authorized means not seizable, and the function reverts `AccountNotSeizable` when the account is authorized. Runtime is correct. The same inverted reading shows up on `SEIZE_ROLE` and `seize_holder_policy_id`, which makes it easy for a later edit to flip the if.

Recommendation: Align the comments on `ensure_seizable`, `SEIZE_ROLE`, and `seize_holder_policy_id` with the invert. State that authorization means the account is not seizable, and that the unset always allow default keeps seizure closed.

Coinbase: Fixed in [PR 4649](#).

Cantina Managed: Fix verified.

3.4.7 Cobalt composite calls and errors are missing from metrics

Severity: Informational.

Context: [metrics.rs#L294-L341](#), [metrics.rs#L443](#), [mod.rs#L20-L33](#), [dispatch.rs#L40-L41](#).

Description: The `BerylMetricLabels::policy_method` mapping omits the five Cobalt policy selectors. Composite validation errors and five additional Cobalt errors from the second pass also fall through to `OtherRevert`. Transaction execution is unchanged, but operational classification is lost for these calls and errors.

Calls using an omitted selector or error are therefore recorded under generic labels, weakening production diagnosis.

Recommendation: Add exhaustive canonical method and error classifier arms, together with selector and error exhaustiveness tests.

Coinbase: Fixed in [PR 4822](#).

Cantina Managed: Fix verified.

3.4.8 Zenith activation is described incorrectly

Severity: Informational.

Context: [provider.rs#L52-L54](#).

Description: The comment at `provider.rs:52` describes Zenith as a placeholder that never activates. The genesis upgrade configuration exposes Zenith as configurable and execution code contains Zenith gated transitions. Current precompile behavior is that Zenith inherits the Cobalt set because it adds no new precompiles. This is a factual documentation defect that can misdirect fork coverage.

The claim can misdirect future fork activation and coverage tests, although the current precompile mapping remains correct.

Recommendation: Describe Zenith as inheriting the Cobalt mapping while separately documenting deployment activation.

Coinbase: Fixed in [PR 4738](#).

Cantina Managed: Fix verified.

3.4.9 Cobalt only wrappers silently select Legacy storage features

Severity: Informational.

Context: [macros.rs#L4-L10](#), [precompile.rs#L8](#), [precompile.rs#L8](#).



Description: The generated wrappers expand through the no feature `base_precompile!` arm whose default is `StorageFeatures::Legacy`. The provider reports Cobalt features elsewhere. Current fixed layouts show no behavior difference, but a future feature sensitive field could inherit the wrong semantics silently.

The risk appears when a future storage field depends on fork features, causing a Cobalt only wrapper to interpret storage under the Legacy feature set.

Recommendation: Make the generated feature explicit and pin it at the EVM boundary. Add a wrapper test for the Cobalt feature.

Coinbase: Fixed in [PR 4835](#).

Cantina Managed: Fix verified.

3.4.10 Public nonce increment helper trusts stale caller state

Severity: Informational.

Context: [eip8130.rs#L993-L1000](#), [storage.rs#L139-L175](#).

Description: `NonceManagerStorage::increment_nonce_from_current` accepts `current` and writes `current + 1`. If storage contains `5` and the caller passes `2`, storage becomes `3`. The current EIP 8130 caller validates the value immediately before consuming it, so no current exploit path was found, but the public API can corrupt nonce progression when reused with stale state.

A future caller that reuses stale state can overwrite a newer nonce and disrupt nonce progression.

Recommendation: Combine validation and increment in the manager or make the unchecked precondition explicit. Add a stale value regression test while retaining the single `SLOAD` optimization.

Coinbase: Fixed in [PR 4748](#).

Cantina Managed: Fix verified.

3.4.11 Direct provider use can omit fork installed precompiles

Severity: Informational.

Context: [provider.rs#L38-L54](#).

Description: `BasePrecompiles::new_with_spec` selects the `Beryl` or `Cobalt` static set, while `install_with_observer` adds `Factory`, `Lookup`, `Registry`, `NonceManager`, and `Transaction Context` entries. A downstream integration using `BasePrecompiles` directly as a `PrecompileProvider` can therefore silently omit those entries. First party builders install the complete map, so this is an integration risk rather than a current production defect.

A downstream integration can expose an incomplete precompile set and fail to route fork installed addresses when it uses the public provider without the recommended builder path.

Recommendation: Remove or rename the incomplete direct provider, store the complete map, or make complete installation mandatory. Add a Cobalt trait boundary test.

Coinbase: Fixed in [PR 4821](#).

Cantina Managed: Fix verified.

3.4.12 Malformed dynamic array lengths can change ABI failure behavior by allocator

Severity: Informational.

Context: [metrics.rs#L590-L597](#), [dispatch.rs#L74-L81](#), [versions.rs#L74-L86](#), [error.rs#L130-L136](#).



Description: The locked `alloy-sol-types` 1.6.1 dynamic array decoder reads the caller supplied length and reserves the output vector before checking that the declared elements fit in calldata. The relevant sequence is `take_offset()`, `raw_child()`, `vec_try_with_capacity(len)`, then `decode_many_from`.

Known selectors in the Policy, Asset, Factory, and common token surfaces pass malformed dynamic arrays to this decoder. For example, a short Policy call can declare `100,000,000 address[]` elements while supplying no element words. The Policy dispatcher charges only the small raw calldata tariff and performs ABI validation before activation or authorization checks.

The project maps the decoder error through `error.to_string()` and returns it as catchable `Abi DecodeFailed` data. The remote differential used identical calldata and showed `Overflow` under one allocator and `Reserve` under a selectively rejecting allocator, producing different revert data lengths. A calling contract could observe that result with `RETURNDATASIZE` or by hashing the bytes.

The behavior is inherited from the target baseline dependency and is not introduced by Cobalt. The Cobalt change adds more affected selectors.

The allocation request can create avoidable memory pressure and can stall or terminate a node if the allocator or operating system refuses the request. If different replay environments return different handled decoder errors, a contract that records the failure result could observe different state or execution output.

The remote review did not demonstrate a confirmed split on the live network and did not establish that current production nodes use divergent allocator policies.

Recommendation: Check the declared length and minimum element size against the remaining calldata before reserving, enforce an aggregate decoder memory budget, and map allocation and structural decode failures to one stable consensus error independent of allocator, operating system, compiler, and dependency diagnostic text. Add identical input tests for normal and allocation rejecting environments.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. The issue is inherited from Alloy 1.6.x, which attempts the vector reservation before validating that all declared elements are present in calldata. Alloy 1.7.x addresses this by checking that the declared dynamic-array length fits in calldata before allocating; the fix will be incorporated through the standard Alloy dependency upgrade.

3.4.13 Static context mutations return catchable reverts instead of exceptional halts

Severity: Informational.

Context: [error.rs#L113-L132](#), [evm.rs#L239-L282](#).

Description: The EVM adapter rejects `SSTORE`, `TSTORE`, `LOG`, and code writes under `is_static` by returning `BasePrecompileError::StaticCallViolation`. `BasePrecompileError::into_precompile_result` maps that variant to `PrecompileOutput::revert` with empty data.

A wrapper can invoke a state changing precompile operation through `STATICCALL` with a caller that otherwise passes the operation's checks, observe an ordinary child failure, and continue execution or commit a later state change. Under the reference EVM, a state changing operation in static context terminates with exceptional halt semantics rather than an ordinary catchable revert.

The attempted state change is rolled back. The difference is the externally observable failure behavior and catchability.

Contracts that depend on reference `STATICCALL` behavior can observe different success handling, return data, gas behavior, and control flow. A wrapper can branch after the native failure and perform later state changes that would not be reached after the corresponding reference exceptional halt.

Recommendation: Map static mutation violations to the same exceptional halt behavior used by the reference EVM instead of returning `PrecompileOutput::revert`. Add production EVM wrapper tests



for static `SSTORE` , `TSTORE` , `LOG` , and code write attempts that compare success status, return data, gas consumption, and state rollback with the reference behavior.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. Aligning `BasePrecompileError::StaticCallViolation` with the reference EVM's exceptional halt semantics would require a backwards-incompatible Base execution-consensus change. The attempted state change is already rolled back; the observable difference is limited to failure class, return data, and gas semantics for callers that inspect the child call result.

3.4.14 Full calldata is copied before gas admission

Severity: Informational.

Context: [dispatch.rs#L47-L55](#), [macros.rs#L13-L30](#), [metrics.rs#L590-L597](#).

Description: The shared `base_precompile!` wrapper copies all call data with `input.data.to_vec()` before the selected dispatcher performs its first gas charge. In the Asset dispatcher, `deduct_calldata_gas` is called only after the copy and provider construction.

A wrapper can copy a large input into EVM memory once, call a warm Base precompile with zero child gas, catch the child `OutOfGas` , and repeat. Each invocation still allocates and copies the complete input before proving that it cannot afford the first calldata word. The path needs no valid selector, ABI arguments, authorization, storage access, or positive child gas.

The retained production EVM integration observed approximately 31.0 milliseconds for 1,000 native calls using a 119,972 byte input versus approximately 17.9 milliseconds for an empty EOA control, while both cases consumed 4,819,880 receipt gas. The execution was a debug and in memory integration, so the timings are diagnostic rather than a production node capacity estimate.

This is distinct from owned ABI expansion and the V1 double decoding issue: the calldata copy occurs before ABI selection and is reachable even with invalid input and zero child gas.

A consensus valid transaction can buy allocator churn and memory bandwidth far below the native work's actual client cost. Repeated calls can slow block execution and derivation and increase node resource pressure. All child calls still return `OutOfGas` , and the review found no direct state divergence or funds impact.

Recommendation: Pass a borrowed calldata slice into dispatch and decode without an unconditional copy. Perform a constant time affordability check before any allocation, copy, provider construction, decoding, formatting, or storage access. If ownership is unavoidable, meter the copy before allocation and gate any gas model change by the appropriate hardfork or version. Add a production EVM regression where a zero gas child with a large reused memory range returns `OutOfGas` without allocating or copying.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. The zero-gas loop is bounded by normal outer EVM `CALL` and memory costs, and all child calls return `OutOfGas` . The `base_precompile!` wrapper's unconditional `input.data.to_vec()` copy will be addressed in the next hardfork release.